

DOCKER

COMMAND REFERENCE

Every command from the learning journey — with description

Container • Image • Network • Volume • Compose • System • Debug

8

categories

100+

commands & flags

1

quick reference

Contents

Container Commands

run, start, stop, restart, rm, ps, exec, logs, inspect, cp, rename, pause, unpause, wait, port, top, stats

Image Commands

build, pull, push, images, rmi, tag, history, inspect, save, load, import, export, prune, commit

Network Commands

network create, ls, inspect, connect, disconnect, rm, prune

Volume Commands

volume create, ls, inspect, rm, prune

Docker Compose Commands

up, down, ps, logs, exec, build, pull, restart, stop, start, run, config, scale

Dockerfile Instructions

FROM, RUN, COPY, ADD, CMD, ENTRYPOINT, ENV, ARG, WORKDIR, USER, EXPOSE, VOLUME, HEALTHCHECK, LABEL

System Commands

system df, prune, info, version, login, logout, builder prune

Debugging Commands

inspect formats, log filters, stats, events, diff, port

```
docker run [OPTIONS] IMAGE [COMMAND]
```

Create a brand new container from an image and start it. The most used Docker command.

-d

Detached — run in background, return terminal immediately

-it

Interactive + TTY — attach terminal, used for shells

--name NAME

Give the container a name for easy reference

-p HOST:CONTAINER

Publish port — e.g. -p 8080:80 maps host:8080 to container:80

-v HOST:CONTAINER

Bind mount — e.g. -v \$(pwd):/app links host dir to container

-v NAME:CONTAINER

Named volume — e.g. -v pgdata:/var/lib/postgresql/data

--network NET

Connect to a specific network — enables DNS by container name

-e KEY=VALUE

Set environment variable inside the container

--env-file FILE

Load environment variables from a file

--rm

Automatically delete container when it stops — great for one-off tasks

--memory=512m

Limit RAM — if exceeded, container is OOMKilled

--cpus=0.5

Limit CPU — 0.5 means half a CPU core

--restart unless-stopped

Restart policy: always restart unless manually stopped

-w /app

Set working directory inside container

--entrypoint bash

Override ENTRYPOINT from Dockerfile

--tmpfs /tmp:size=512m

Mount a RAM-backed temporary filesystem

-P

Publish ALL EXPOSEd ports to random host ports

--privileged

Give container full host access — use with extreme caution

--user 1001

Run as a specific user ID

```
docker start CONTAINER
```

Start an existing STOPPED container. Resumes the same writable layer — data intact.

docker start = WAKE UP a house you already own. Use after docker stop, not docker run.

```
docker stop CONTAINER
```

Gracefully stop a running container. Sends SIGTERM, waits 10s, then SIGKILL.

-t 30

Wait 30 seconds before force-killing (default is 10)

`docker restart CONTAINER`

Stop then start a container. Equivalent to `docker stop + docker start`.

`docker rm CONTAINER`

Delete a stopped container permanently. The writable layer is gone forever.

-f

Force remove even if container is running (sends SIGKILL first)

-v

Also remove anonymous volumes attached to the container

docker rm deletes the writable layer. Named volumes are NOT affected.

`docker ps`

List all currently RUNNING containers.

-a

Show ALL containers — running AND stopped

-q

Quiet mode — only show container IDs (useful for scripting)

--format

Custom output format e.g. `--format 'table {{.Names}}\t{{.Status}}'`

-n 5

Show last 5 containers created

-l

Show the most recently created container

`docker exec [OPTIONS] CONTAINER COMMAND`

Run a command inside an ALREADY RUNNING container. Does not create a new container.

-it

Interactive + TTY — required for shell sessions

-e KEY=VALUE

Set env variable for this exec session only

-w /path

Set working directory for this command

-u user

Run as specific user

docker exec = VISIT someone in their house. Container keeps running when you exit.

`docker logs CONTAINER`

Fetch stdout and stderr output from a container.

-f

Follow — stream logs in real time like `tail -f`

--tail 100

Show only the last 100 lines

-t

Show timestamps on each log line

--since 30m

Show logs from last 30 minutes

--since 2024-01-01T10:00:00

Show logs since a specific timestamp

2>&1 | grep ERROR

Filter logs for errors (pipe to `grep`)

```
docker inspect CONTAINER|IMAGE|NETWORK|VOLUME
```

Return detailed JSON metadata about any Docker object.

--format '{{.State.Status}}'

Extract a specific field — running/stopped/paused

--format '{{.State.Pid}}'

Get the host PID of the container process

--format '{{json .NetworkSettings.Networks}}'

Get network info as JSON

--format '{{.State.OOMKilled}}'

Check if container was killed by out-of-memory

--format '{{json .GraphDriver}}'

See overlay2 layer paths (Linux only, null on Mac)

--format '{{.Mounts}}'

See all volume and bind mount info

--format '{{.Config.Env}}'

See all environment variables

```
docker cp SRC DEST
```

Copy files between host and container in either direction.

docker cp ./file.txt mycontainer:/app/

Copy from host INTO container

docker cp mycontainer:/app/log.txt ./

Copy from container TO host

```
docker rename OLD_NAME NEW_NAME
```

Rename a container. Useful when you created a container without --name.

```
docker pause CONTAINER
```

Freeze all processes in a container using SIGSTOP. Container still exists in memory.

```
docker unpause CONTAINER
```

Resume a paused container. All processes continue from where they froze.

```
docker port CONTAINER
```

Show all port mappings for a container.

Output: 80/tcp -> 0.0.0.0:8080 means host:8080 → container:80

```
docker top CONTAINER
```

Display the running processes inside a container, like ps aux.

```
docker stats [CONTAINER]
```

Live stream of CPU, memory, network, and disk usage.

--no-stream

Show one snapshot then exit — no live streaming

--format

Custom format for output

```
docker wait CONTAINER
```

Block until a container stops, then print its exit code. Useful in scripts.

```
docker diff CONTAINER
```

Show filesystem changes made by the container since it started. A=added, C=changed, D=deleted.

```
docker commit CONTAINER IMAGE:TAG
```

Create a new image from a container's current state. Avoid in production — use Dockerfile instead.

-m 'message'

Commit message

-a 'author'

Author name

docker build [OPTIONS] PATH

Build an image from a Dockerfile. PATH is the build context sent to Docker daemon.

-t name:tag

Tag the built image — e.g. -t myapp:v1.0.0

-t name:tag -t name:latest

Apply multiple tags in one build

-f Dockerfile.prod

Use a different Dockerfile — useful for multi-environment

--no-cache

Ignore all cached layers — force complete rebuild

--build-arg KEY=VAL

Pass ARG values defined in Dockerfile

--target stage

Stop at a specific stage in a multi-stage build

--secret id=KEY,src=FILE

Pass secret file without baking it into layers

--platform linux/amd64

Build for a specific architecture

--progress plain

Show full build output instead of compressed progress

2>&1 | head -3

Check build context size — look at first line of output

docker pull IMAGE[:TAG]

Download an image from a registry (Docker Hub by default).

--platform linux/arm64

Pull for a specific architecture

-a

Pull all tagged versions of the image

docker push IMAGE[:TAG]

Upload a local image to a registry. Must be tagged with registry path first.

--all-tags

Push all tags of the image

Must run docker login first for private registries or Docker Hub.

docker images

List all images stored locally.

-a

Show all images including intermediate layers

-q

Show only image IDs

--no-trunc

Show full image IDs without truncating

--filter dangling=true

Show only dangling (untagged) images

--format

Custom output format

```
docker rmi IMAGE[:TAG]
```

Remove one or more images from local storage.

-f

Force remove even if containers are using it

\$(docker images -q)

Remove ALL images — pipe from docker images -q

```
docker tag SOURCE_IMAGE TARGET_IMAGE
```

Create a new tag pointing to an existing image. Used before pushing to a registry.

Example: docker tag myapp:latest myuser/myapp:v1.0.0 — prepares for docker push

```
docker history IMAGE
```

Show the history of an image — every layer, its size, and the command that created it.

--no-trunc

Show full commands without truncating

-q

Show only layer IDs

WARNING: docker history exposes all ENV values ever set. Never put secrets in ENV.

```
docker image inspect IMAGE
```

Show detailed metadata about an image including layers, env vars, exposed ports.

```
docker save IMAGE -o file.tar
```

Export an image to a tar archive file. Used for air-gapped transfers.

```
docker load -i file.tar
```

Load an image from a tar archive created by docker save.

```
docker image prune
```

Remove all dangling images — untagged images with no container using them.

-a

Remove ALL unused images — not just dangling ones

-f

Skip confirmation prompt

```
docker image ls --filter reference='myapp*'
```

List images matching a pattern. Useful for cleaning up old versions.

■ NETWORK COMMANDS

[docker network commands ...](#)

```
docker network create NETWORK
```

Create a custom bridge network with embedded DNS. Containers on same network find each other by name.

--driver bridge

Bridge driver — single host networking (default)

--driver overlay

Overlay driver — multi-host networking (requires Swarm)

--subnet 192.168.10.0/24

Specify the IP subnet for this network

--gateway 192.168.10.1

Specify the gateway IP

--attachable

Allow standalone containers to connect (overlay only)

Always use custom networks instead of the default bridge — you get DNS by container name.

```
docker network ls
```

List all networks on the host — bridge, host, none, and any custom networks.

```
docker network inspect NETWORK
```

Show detailed info about a network — connected containers, subnet, gateway, driver.

```
docker network connect NETWORK CONTAINER
```

Connect a running container to an additional network. A container can be on multiple networks.

```
docker network disconnect NETWORK CONTAINER
```

Remove a container from a network. Container keeps running.

```
docker network rm NETWORK
```

Delete a network. All containers must be disconnected first.

```
docker network prune
```

Remove all networks not used by any container.

-f

Skip confirmation prompt

■ VOLUME COMMANDS

[docker volume commands ...](#)

```
docker volume create NAME
```

Create a named volume. Docker manages the storage at `/var/lib/docker/volumes/NAME/_data`.

```
docker volume ls
```

List all named volumes on the host.

--filter dangling=true

Show only volumes not attached to any container

```
docker volume inspect NAME
```

Show volume details — Mountpoint shows where data actually lives on the host.

```
docker volume rm NAME
```

Delete a named volume permanently. Data is gone.

```
docker volume prune
```

Remove all volumes not used by any container.

-f

Skip confirmation

```
# Backup a named volume to a tar file
```

Copy volume data to host via a temporary container.

```
docker run --rm -v pgdata:/source:ro -v $(pwd):/backup ubuntu tar czf /backup/pgdata.tar.gz  
-C /source .
```

Backup

```
docker run --rm -v pgdata:/target -v $(pwd):/backup ubuntu tar xzf /backup/pgdata.tar.gz -C  
/target
```

Restore

docker compose up

Start all services defined in compose.yml. Creates containers, networks, volumes.

-d

Detached — run all services in background

--build

Rebuild images before starting — use after Dockerfile changes

--force-recreate

Recreate containers even if config hasn't changed

--no-deps SERVICE

Start only this service without its dependencies

--scale web=3

Run 3 instances of the web service

docker compose down

Stop and remove all containers and networks created by compose up.

-v

Also remove named volumes — WARNING: deletes all data

--rmi all

Also remove all images used by services

--remove-orphans

Remove containers for services no longer in compose.yml

docker compose ps

Show status of all services — running, stopped, health status.

docker compose logs

Show output from all services.

-f

Follow — stream logs in real time

SERVICE_NAME

Show logs for one specific service only

--tail 50

Show only last 50 lines per service

-t

Show timestamps

docker compose exec SERVICE COMMAND

Run a command inside a running service container. Same as docker exec but uses service name.

docker compose exec api bash

Open a shell in the api service

docker compose exec db psql -U postgres

Open postgres shell

```
docker compose build
```

Build or rebuild images for services that have a 'build' section in compose.yml.

--no-cache

Build without cache

SERVICE

Build only a specific service

```
docker compose pull
```

Pull latest images for all services using pre-built images.

```
docker compose restart SERVICE
```

Restart one or all services without rebuilding. Picks up env var changes.

```
docker compose stop SERVICE
```

Stop running service containers without removing them.

```
docker compose start SERVICE
```

Start stopped service containers without recreating them.

```
docker compose run SERVICE COMMAND
```

Run a one-off command in a new container for a service. Container removed after.

--rm

Remove container after command finishes (default for run)

docker compose run api python migrate.py

Run database migration

```
docker compose config
```

Validate compose.yml and print the resolved config with all env variable substitutions applied.

```
docker compose top
```

Show processes running inside each service container.

FROM image:tag [AS stage_name]

Set the base image. Every Dockerfile must start with FROM. AS names the stage for multi-stage builds.

FROM python:3.11.9-slim

Always pin to exact version — never use :latest in production

FROM python:3.11-slim AS builder

Named stage — reference with COPY --from=builder

FROM scratch

Empty base — used for Go static binaries, nothing else

FROM python@sha256:abc...

Digest pin — immune to tag rewrites

RUN command

Execute a command during build. Each RUN creates a new layer. Use && to chain commands.

RUN apt-get update && apt-get install -y pkg && rm -rf /var/lib/apt/lists/*

Correct apt pattern — cleanup in SAME RUN

RUN pip install --no-cache-dir -r requirements.txt

No-cache pip install

RUN --mount=type=secret,id=pip pip install ...

BuildKit secret mount — never in image layers

Cleanup MUST be in the same RUN instruction. A separate RUN rm cannot remove bytes from a previous layer.

COPY [--chown=user:group] [--from=stage] SRC DEST

Copy files from build context or another build stage into the image.

COPY requirements.txt .

Copy only requirements first — enables pip cache layer

COPY . .

Copy all source code — put this LAST for best caching

COPY --chown=appuser:appgroup . .

Set file ownership — prevents permission denied errors

COPY --from=builder /app/build/*.so /app/

Multi-stage — copy artifacts from builder stage

CMD ["executable", "arg1", "arg2"]

Default command when container starts. Completely replaced if you type anything after image name in docker run.

CMD ["python", "main.py"]

Exec form — correct. python is PID 1.

CMD python main.py

Shell form — WRONG. /bin/sh is PID 1, app misses SIGTERM.

Always use JSON array (exec form). Shell form breaks graceful shutdown.

ENTRYPOINT ["executable", "arg1"]

Fixed executable that always runs. Arguments typed after image name are APPENDED, not replacing.

ENTRYPOINT ["python", "serve.py"]

Container IS a python server — cannot easily override

CMD ["--port", "8080"]

Combined with ENTRYPOINT: CMD provides default args, you can override at runtime

ENTRYPOINT = job title (fixed). CMD = default task (overrideable). Together = best production pattern.

ENV KEY=VALUE

Set environment variable. Persists into the running container. Visible in docker history and inspect.

ENV PYTHONUNBUFFERED=1

Python prints to stdout immediately — critical for Docker logs

ENV PYTHONDONTWRITEBYTECODE=1

Do not write .pyc files inside container

ENV APP_PORT=8080

Default port — override at runtime with docker run -e APP_PORT=9090

NEVER put secrets in ENV. They are permanently visible in docker history to anyone with image access.

ARG KEY[=default]

Build-time variable only. Gone after build completes — NOT available in running container.

ARG PYTHON_VERSION=3.11

Define with default

FROM python:\${PYTHON_VERSION}-slim

Use ARG in FROM instruction

docker build --build-arg PYTHON_VERSION=3.12 .

Pass value at build time

WORKDIR /path

Set working directory for all subsequent RUN, COPY, CMD, ENTRYPOINT instructions. Creates dir if missing.

Never use RUN cd /path — the directory change dies with the shell. WORKDIR persists across all instructions.

USER username|uid

Switch to a non-root user for subsequent instructions and the running container.

RUN useradd --uid 1001 --create-home appuser

Create the user first

USER appuser

Switch — everything after runs as appuser

Always switch to non-root before CMD. Install deps as root, then USER appuser, then CMD.

EXPOSE port[/protocol]

Document which port the container listens on. Does NOT publish the port. Use -p in docker run to actually open it.

EXPOSE 8080

TCP port (default)

EXPOSE 5353/udp

UDP port

VOLUME ["/path"]

Create a mount point for a named volume. Rarely used directly — prefer -v in docker run or compose volumes.

HEALTHCHECK --interval=30s --timeout=10s --retries=3 CMD command

Define a health check command. Docker runs it periodically — container becomes healthy/unhealthy.

CMD curl -f http://localhost:8080/health || exit 1

HTTP health check

CMD-SHELL pg_isready -U postgres

Postgres health check

--start-period=30s

Grace period on startup before failures count

```
LABEL key=value
```

Add metadata to an image — version, maintainer, description. Queryable with `docker inspect`.

```
ADD src dest
```

Like `COPY` but also auto-extracts tar files and supports URLs. Prefer `COPY` unless you need those features.

Use `COPY` instead of `ADD` for predictability. `ADD`'s auto-extraction can cause unexpected results.

```
docker system df
```

Show disk usage broken down by images, containers, volumes, and build cache.

-v

Verbose — list every object individually with its size

Run this BEFORE any prune command to understand what you have and what is reclaimable.

```
docker system prune
```

Remove all stopped containers, dangling images, unused networks, and build cache.

-a

Also remove images not referenced by any container — much more aggressive

-f

Skip confirmation prompt

--volumes

Also remove unused volumes — WARNING: data loss

Always run docker system df first to understand the impact before pruning.

```
docker builder prune
```

Remove build cache only. Safe to run — never affects images, containers, or volumes.

-a

Remove all build cache including cache still in use by images

-f

Skip confirmation

Safest cleanup command. Run this first when you need to reclaim disk space.

```
docker system info
```

Show system-wide Docker information — containers, images, storage driver, OS, memory.

```
docker version
```

Show Docker client and server version information.

```
docker login [REGISTRY]
```

Log in to Docker Hub or a private registry. Required before docker push.

docker login

Log in to Docker Hub

docker login registry.example.com

Log in to private registry

docker login --username AWS --password-stdin REGISTRY_URL

AWS ECR login pattern

```
docker logout [REGISTRY]
```

Log out from a registry. Removes stored credentials.

`docker events`

Stream real-time events from Docker daemon — container start/stop, image pull, etc.

`--filter event=die`

Only show container death events

`--since 30m`

Events from last 30 minutes

```
# The debugging ladder – run in this order
```

When something breaks, run these commands in sequence.

```
docker ps -a
```

1. Is the container running? What is its STATUS?

```
docker logs --tail 50 mycontainer
```

2. What did it print? Error is almost always here.

```
docker inspect mycontainer --format '{{.State.Status}}'
```

3. Is it healthy/unhealthy/restarting?

```
docker exec -it mycontainer bash
```

4. Get inside and look around.

```
docker exec -it mycontainer bash
```

Enter a running container interactively. Most important debugging command.

```
docker exec -it mycontainer sh
```

Use sh if bash is not installed (alpine images)

```
docker run -it --entrypoint bash myimage
```

Enter image that won't start — override ENTRYPOINT

```
docker run --rm -it myimage bash
```

Throwaway container for exploring image contents

```
docker run --network container:CONTAINER nicolaka/netshoot
```

Attach a container full of network debugging tools to a running container's network namespace.

netshoot has: ping, curl, dig, nslookup, tcpdump, netstat, ss, ip, traceroute — all available.

```
docker inspect CONTAINER --format '{{json .State}}'
```

See full state of a container — Status, Pid, ExitCode, OOMKilled, StartedAt, FinishedAt.

```
docker inspect CONTAINER --format '{{.NetworkSettings.Networks}}'
```

See which networks a container is connected to and its IP address on each.

```
docker inspect CONTAINER --format '{{.Config.Env}}'
```

List all environment variables inside a container — from Dockerfile ENV and docker run -e flags.

```
docker inspect CONTAINER --format '{{.State.Pid}}'
```

Get the host PID of the container's main process. Proves container is just a process.

Then run: ls -la /proc/PID/ns to see the 6 namespace files Docker created.

```
docker logs -f mycontainer 2>&1 | grep -i error
```

Stream logs and filter for errors. 2>&1 merges stderr into stdout so grep catches both.

```
docker stats --no-stream
```

One-time snapshot of all containers' CPU, memory, network, disk usage. No live streaming.

```
docker port CONTAINER
```

Show all published port mappings. Output: 80/tcp -> 0.0.0.0:8080 means host:8080 → container:80.

```
docker diff CONTAINER
```

Show all filesystem changes since container started. A=Added, C=Changed, D=Deleted.

```
docker exec mycontainer cat /etc/resolv.conf
```

Check DNS config inside container. Should show nameserver 127.0.0.11 on custom networks.

127.0.0.11 is Docker's embedded DNS. If missing, container is on wrong network.

```
docker exec mycontainer nslookup other-container
```

Test DNS resolution by name inside the container. Diagnoses network connection issues.

Common Error Reference

Error / Symptom	Most Likely Cause	Fix
Port is already allocated	Something on host already uses that port	Change -p HOST_PORT:CONTAINER, or stop conflicting service
No such file or directory (COPY)	File not in build context or path wrong	Check .dockerignore, verify file exists, check relative path
Name or service not known	Containers on different networks or wrong name	Put both containers on same custom network with <code>docker network create</code>
Connection refused	App not listening on 0.0.0.0 or port not published	Add <code>--host 0.0.0.0</code> to server, check -p flag
OOMKilled: true	Container exceeded memory limit	Increase <code>--memory</code> limit or fix memory leak
Exit code 1	App crashed on startup	<code>docker logs mycontainer</code> — error is always there
image not found	Wrong image name, not pulled, not logged in	<code>docker pull</code> first, check spelling, <code>docker login</code> for private
Permission denied	App writing to dir owned by root	Add <code>--chown</code> to COPY, check USER instruction
GraphDriver returns null	Running on Mac — paths live inside Docker Desktop VM	Normal on Mac — use Linux VM or server to see overlay2 paths

Quick Reference Cheat Sheet

LIFECYCLE

<code>docker run -d --name app -p 80:80 myimage</code>	Create+start container in background
<code>docker stop app</code>	Gracefully stop
<code>docker start app</code>	Resume stopped container
<code>docker restart app</code>	Stop then start
<code>docker rm -f app</code>	Force delete running container
<code>docker run --rm -it myimage bash</code>	Throwaway interactive container

INSPECT & DEBUG

<code>docker ps -a</code>	All containers with status
<code>docker logs -f app</code>	Stream live logs
<code>docker exec -it app bash</code>	Enter running container
<code>docker inspect app</code>	Full JSON metadata
<code>docker stats --no-stream</code>	CPU/RAM snapshot
<code>docker port app</code>	Show port mappings
<code>docker diff app</code>	Show filesystem changes

IMAGES

<code>docker build -t myapp:v1 .</code>	Build and tag image
<code>docker images</code>	List local images
<code>docker pull nginx:1.25</code>	Download image
<code>docker push myuser/myapp:v1</code>	Upload to registry
<code>docker rmi myapp:old</code>	Delete image
<code>docker history myimage</code>	Show layers and sizes
<code>docker image prune -a</code>	Remove unused images

NETWORKS

<code>docker network create mynet</code>	Create custom bridge network
<code>docker network ls</code>	List all networks
<code>docker network inspect mynet</code>	Show connected containers
<code>docker network connect mynet app</code>	Add container to network
<code>docker network prune</code>	Remove unused networks
<code>docker exec app cat /etc/resolv.conf</code>	Check DNS inside container

VOLUMES

<code>docker volume create pgdata</code>	Create named volume
<code>docker volume ls</code>	List volumes
<code>docker volume inspect pgdata</code>	Show mountpoint on host
<code>docker volume rm pgdata</code>	Delete volume (data gone)
<code>docker volume prune</code>	Remove unused volumes
<code>-v pgdata:/var/lib/postgresql/data</code>	Mount named volume in docker run

SYSTEM

<code>docker system df</code>	Show disk usage by category
<code>docker system df -v</code>	Verbose — list every object

<code>docker builder prune</code>	Remove build cache (safest)
<code>docker system prune</code>	Remove stopped containers+dangling images
<code>docker system prune -a</code>	Remove everything unused
<code>docker system info</code>	Docker engine info